

Correctness and running time of Huffman's algorithm

Vassos Hadzilacos

We prove the correctness of Huffman's algorithm by induction on the number of symbols n in the alphabet.

The base case, $n = 2$ is obvious because the only possibility (that is not obviously suboptimal) is a code where both codewords are one bit long, which is what Huffman's algorithm produces in this case.

Suppose that the algorithm produces an optimal tree for alphabets with $n - 1 \geq 2$ symbols and their associated frequencies. We will prove that it produces an optimal tree for alphabets with n symbols and their associated frequencies.

Let Γ be an alphabet with n symbols, and $f(a)$ be the frequency for each $a \in \Gamma$. Let H be the tree produced by Huffman's algorithm for Γ, f . We must prove that H is optimal for this input.

By the algorithm, there are two symbols of minimum frequency (according to f) that are siblings in H ; let these symbols be x and y . Let z be a new symbol (that is not in Γ); and let $\Gamma' = (\Gamma - \{x, y\}) \cup \{z\}$ and f' be frequencies of the symbols in Γ' defined by

$$f'(a) = \begin{cases} f(a), & \text{if } a \neq z \\ f(x) + f(y), & \text{if } a = z. \end{cases}$$

(Intuitively, we are replacing the symbols x and y with a new symbol z , whose frequency is the sum of the frequencies of x and y .) Finally, let H' be the tree obtained from H by removing x and y and replacing their parent by z . From the definition of weighted average depth, we have

$$\mathbf{ad}(H) = \mathbf{ad}(H') + (f(x) + f(y)). \quad (1)$$

Note that H' is a tree produced by Huffman's algorithm on input Γ', f' . Γ' has $n - 1$ symbols so, by induction hypothesis,

$$H' \text{ is optimal for } \Gamma', f'. \quad (2)$$

Now, let T be an optimal tree for Γ, f . Without loss of generality, we can assume that x and y are siblings and are at maximum depth in T . (If not, we can move them so that they are siblings at the maximum depth of T without increasing the weighted average depth of the tree, by swapping them with symbols that are siblings at the maximum depth.) Let T' be obtained from T as H' was obtained from H . Thus, T' is a tree for Γ', f' . We have:

$$\begin{aligned} \mathbf{ad}(T) &= \mathbf{ad}(T') + (f(x) + f(y)) && [\text{by definition of } \mathbf{ad}] \\ &\geq \mathbf{ad}(H') + (f(x) + f(y)) && [\text{by (2)}] \\ &= \mathbf{ad}(H) && [\text{by (1)}] \end{aligned}$$

Since T is optimal for Γ, f , so is H . So, Huffman's algorithm produces optimal trees for alphabets with n symbols and their associated frequencies.

We can implement this algorithm to run in $O(n \log n)$ time using heaps. Let n be the number of symbols in the alphabet, and $f(i)$ be the frequency of the i -th symbol, $1 \leq i \leq n$. The algorithm constructs a full

binary tree with $2n - 1$ nodes, each labeled with a positive integer i , $1 \leq i \leq 2n - 1$. Nodes labeled $1, 2, \dots, n$ are leaves, where the leaf node labeled i corresponds to the i -th symbol. Nodes $n + 1, n + 2, \dots, 2n - 1$ are internal nodes, i.e., nodes that are not leaves. (Note that a **full** binary tree with n leaves has $n - 1$ internal nodes, and therefore a total of $2n - 1$ nodes. This is easy to prove by complete induction.)

The algorithm uses a heap H that stores pairs of the form $x = (i, p)$ where $1 \leq i \leq 2n - 1$ and $0 \leq p \leq 1$. The first component of the pair x , denoted $x.\text{label}$, is the label of a node in the tree that the algorithm constructs. The second component, denoted $x.\text{freq}$, is the sum of the frequencies of all the symbols stored in the leaves of the subtree rooted at the node labeled $x.\text{label}$; $x.\text{freq}$ is used as the priority for ordering the pairs in the heap H . The algorithm expressed in pseudocode is shown below.

```

HUFFMAN( $n, f$ )
1  for  $i := 1$  to  $n$  do
2       $H[i] := (i, f(i))$ 
3      create a leaf node labeled  $i$  (both children are NIL)
4  BUILDHEAP( $H$ )
5  for  $i := n + 1$  to  $2n - 1$  do
6       $x := \text{EXTRACTMIN}(H); y := \text{EXTRACTMIN}(H)$ 
7      create a node labeled  $i$  with children the nodes labeled  $x.\text{label}$  and  $y.\text{label}$ 
8      INSERT( $H, (i, x.\text{freq} + y.\text{freq})$ )

```

This algorithm runs in $O(n \log n)$ time: Putting the first n pairs into H and creating the n leaves takes $O(n)$ time (lines 1–3), and turning H into a heap using BUILDHEAP also takes $O(n)$ time (line 4). The **for** loop in lines 5–8 is repeated $n - 1$ times. In each iteration we perform two EXTRACTMIN operations and one INSERT operation, each of which takes $O(\log n)$ time. So the loop takes $O(n \log n)$ time, and the entire algorithm takes $O(n) + O(n) + O(n \log n) = O(n \log n)$ time.

Lecture 17: Huffman Coding

CLRS- 16.3

Outline of this Lecture

- Codes and Compression.
- Huffman coding.
- Correctness of the Huffman coding algorithm.

Suppose that we have a 100,000 character data file that we wish to store. The file contains only 6 characters, appearing with the following frequencies:

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
Frequency in '000s	45	13	12	16	9	5

A *binary code* encodes each character as a binary string or *codeword*. We would like to find a binary code that encodes the file using as few bits as possible, ie., *compresses it* as much as possible.

In a *fixed-length code* each codeword has the same length. In a *variable-length code* codewords may have different lengths. Here are examples of fixed and variable length codes for our problem (note that a fixed-length code must have at least 3 bits per codeword).

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
Freq in '000s	45	13	12	16	9	5
a fixed-length	000	001	010	011	100	101
a variable-length	0	101	100	111	1101	1100

The *fixed length-code* requires 300,000 bits to store the file. The *variable-length code* uses only

$$(5 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1000 = 224,000 \text{ bits,}$$

saving a lot of space! Can we do better?

Note: In what follows a *code* will be a set of codewords, e.g.,
 {000, 001, 010, 011, 100, 101}
 and {0, 101, 100, 111, 1101, 1100}

Encoding

Given a code (corresponding to some alphabet Γ) and a message it is easy to *encode* the message. Just replace the characters by the codewords.

Example: $\Gamma = \{a, b, c, d\}$

If the code is

$$C_1\{a = 00, b = 01, c = 10, d = 11\}.$$

then **bad** is encoded into **010011**

If the code is

$$C_2 = \{a = 0, b = 110, c = 10, d = 111\}$$

then **bad** is encoded into **1101111**

Decoding

$$C_1 = \{a = 00, b = 01, c = 10, d = 11\}.$$

$$C_2 = \{a = 0, b = 110, c = 10, d = 111\}.$$

$$C_3 = \{a = 1, b = 110, c = 10, d = 111\}$$

Given an encoded message, *decoding* is the process of turning it back into the original message.

A message is *uniquely decodable* (vis-a-vis a particular code) if it can only be decoded in one way.

For example relative to C_1 , **010011** is uniquely decodable to **bad**.

Relative to C_2 **1101111** is uniquely decodable to **bad**.

But, relative to C_3 , **1101111** is **not** uniquely decipherable since it could have encoded either **bad** or **acad**.

In fact, one can show that every message encoded using C_1 and C_2 are uniquely decipherable. The unique decipherability property is needed in order for a code to be useful.

Prefix-Codes

Fixed-length codes are always uniquely decipherable (why).

We saw before that these do not always give the best compression so we prefer to use variable length codes.

Prefix Code: A code is called a **prefix (free) code** if no codeword is a prefix of another one.

Example: $\{a = 0, b = 110, c = 10, d = 111\}$ is a prefix code.

Important Fact: Every message encoded by a prefix free code is uniquely decipherable. Since no codeword is a prefix of any other we can always find the first codeword in a message, peel it off, and continue decoding. Example:

$$01101100 = 01101100 = abba$$

We are therefore interested in finding good (best compression) prefix-free codes.

Fixed-Length versus Variable-Length Codes

Problem: Suppose we want to store messages made up of 4 characters a, b, c, d with frequencies 60, 5, 30, 5 (percents) respectively. What are the fixed-length codes and prefix-free codes that use the least space?

Fixed-Length versus Variable-Length Prefix Codes

Solution:

characters	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
frequency	60	5	30	5
fixed-length code	00	01	10	11
prefix code	0	110	10	111

To store 100 of these characters,

(1) the fixed-length code requires $100 \times 2 = 200$ bits,

(2) the prefix code uses only

$$60 \times 1 + 5 \times 3 + 30 \times 2 + 5 \times 3 = 150$$

a 25% saving.

Remark: We will see later that this is the *optimum* (lowest cost) prefix code.

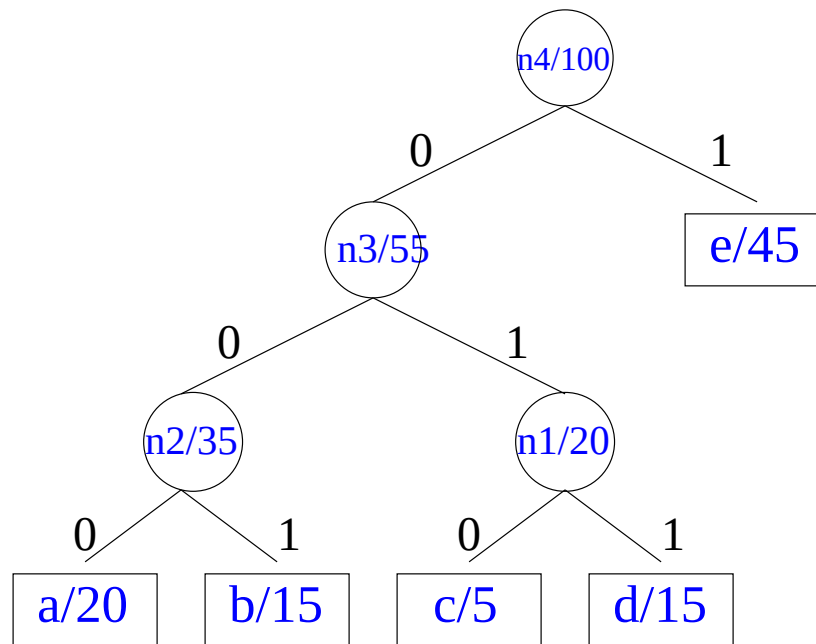
Optimum Source Coding Problem

The problem: Given an **alphabet** $A = \{a_1, \dots, a_n\}$ with **frequency distribution** $f(a_i)$ find a binary prefix code C for A that minimizes the number of bits

$$B(C) = \sum_{a=1}^n f(a_i) L(c(a_i))$$

needed to encode a message of $\sum_{a=1}^n f(a)$ characters, where $c(a_i)$ is the codeword for encoding a_i , and $L(c(a_i))$ is the length of the codeword $c(a_i)$.

Remark: Huffman developed a nice greedy algorithm for solving this problem and producing a minimum-cost (optimum) prefix code. The code that it produces is called a **Huffman code** .



$\{a = 000, b = 001, c = 010, d = 011, e = 1\}$

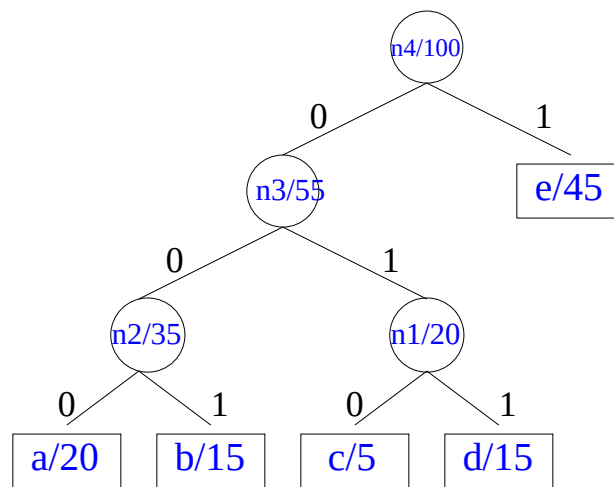
Correspondence between Binary Trees and prefix codes.

1-1 correspondence between leaves and characters.

Label of leaf is frequency of character.

Left edge is labelled 0; right edge is labelled 1

Path from root to leaf is codeword associated with character.



$\{a = 000, b = 001, c = 010, d = 011, e = 1\}$

Note that $d(a_i)$, the depth of leaf a_i in tree T is equal to the length, $L(c(a_i))$ of the codeword in code C associated with that leaf. So,

$$\sum_{a=1}^n f(a_i) L(c(a_i)) = \sum_{a=1}^n f(a_i) d(a_i).$$

The sum $\sum_{a=1}^n f(a_i) d(a_i)$ is the *weighted external pathlength* of tree T .

The Huffman encoding problem is equivalent to the **minimum-weight external pathlength problem**: given weights $f(a_1), \dots, f(a_n)$, find tree T with n leaves labelled $a_1, \dots, a_n$ that has minimum weighted external path length.

Huffman Coding

Step 1: Pick two letters x, y from alphabet A with the smallest frequencies and create a subtree that has these two characters as leaves. (greedy idea)
Label the root of this subtree as z .

Step 2: Set frequency $f(z) = f(x) + f(y)$.
Remove x, y and add z creating new alphabet $A' = A \cup \{z\} - \{x, y\}$.
Note that $|A'| = |A| - 1$.

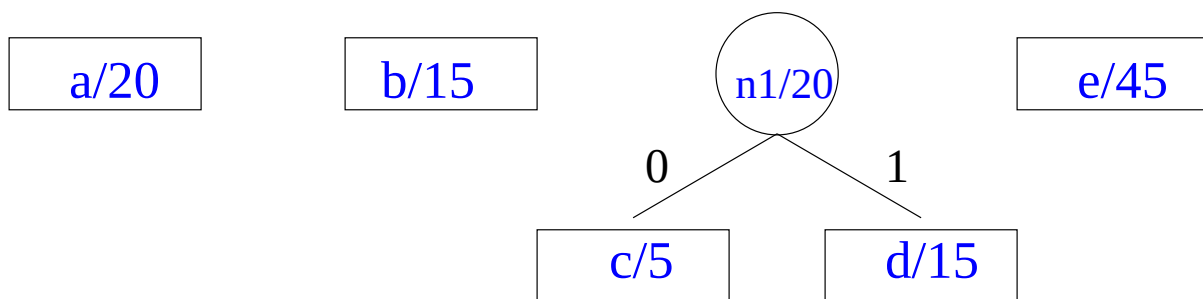
Repeat this procedure, called *merge*, with new alphabet A' until an alphabet with only one symbol is left.

The resulting tree is the **Huffman code**.

Example of Huffman Coding

Let $A = \{a/20, b/15, c/5, d/15, e/45\}$ be the alphabet and its frequency distribution.

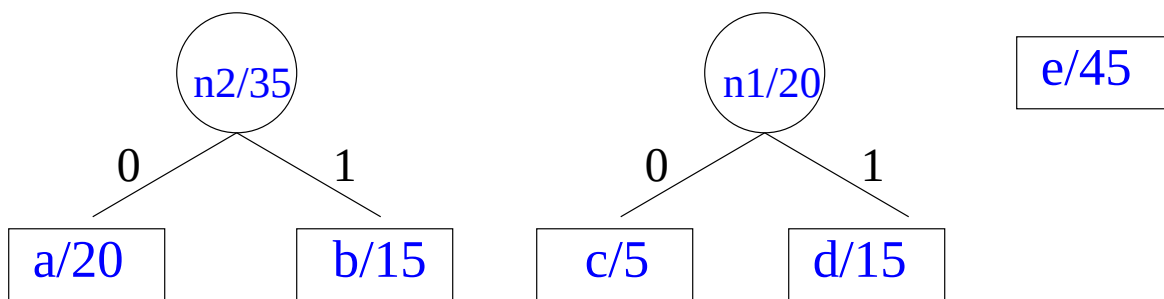
In the first step Huffman coding merges c and d .



Alphabet is now $A_1 = \{a/20, b/15, n1/20, e/45\}$.

Example of Huffman Coding – Continued

Alphabet is now $A_1 = \{a/20, b/15, n1/20, e/45\}$.
Algorithm merges a and b
(could also have merged $n1$ and b).

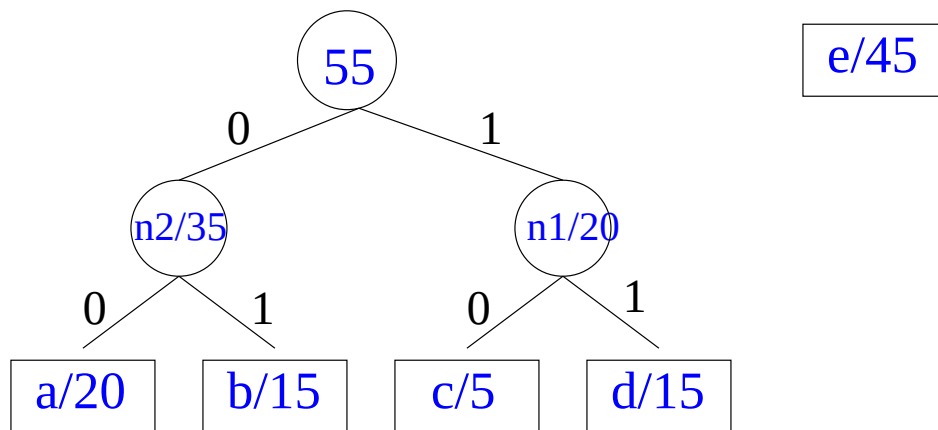


New alphabet is $A_2 = \{n2/35, n1/20, e/45\}$.

Example of Huffman Coding – Continued

Alphabet is $A_2 = \{n2/35, n1/20, e/45\}$.

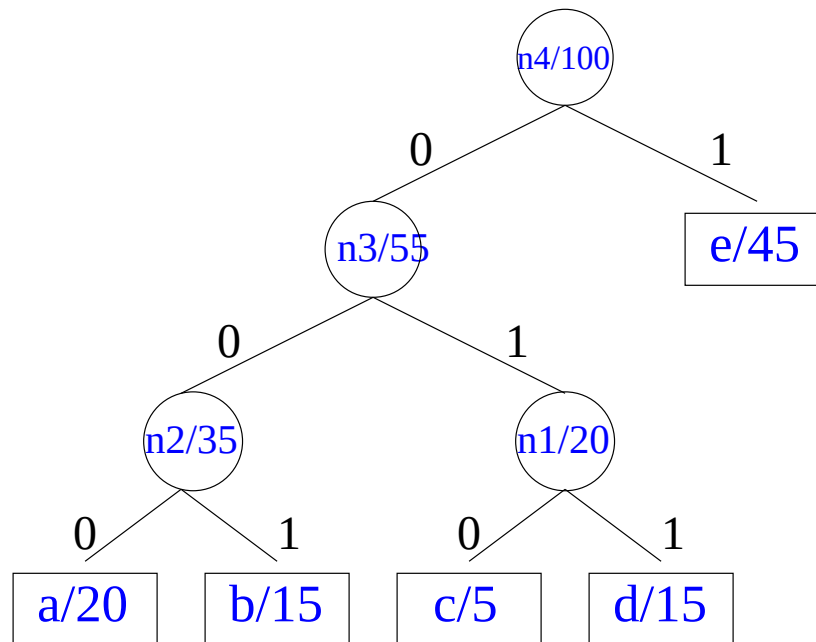
Algorithm merges $n1$ and $n2$.



New alphabet is $A_3 = \{n3/55, e/45\}$.

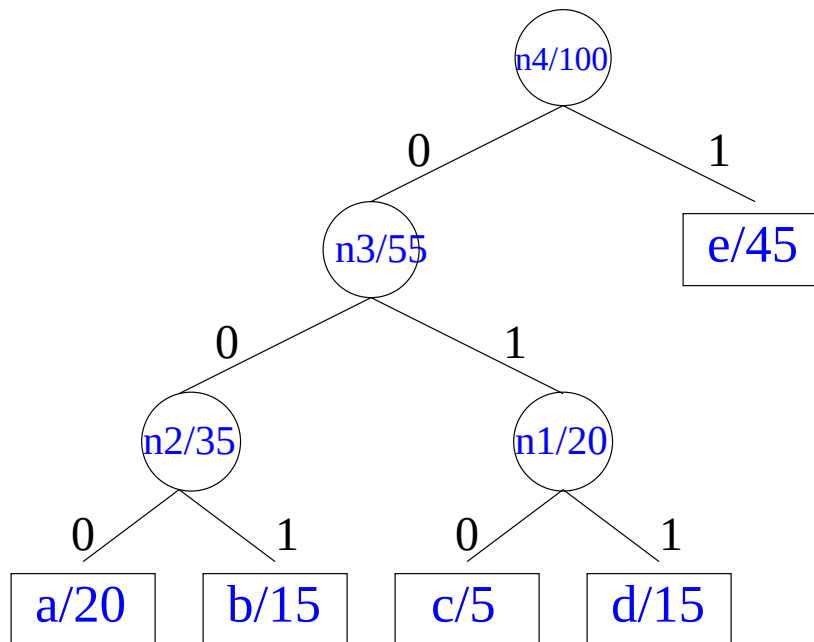
Example of Huffman Coding – Continued

Current alphabet is $A_3 = \{n3/55, e/45\}$.
Algorithm merges e and $n3$ and finishes.



Example of Huffman Coding – Continued

Huffman code is obtained from the Huffman tree.



Huffman code is

$a = 000$, $b = 001$, $c = 010$, $d = 011$, $e = 1$.

This is the optimum (minimum-cost) prefix code for this distribution.

Huffman Coding Algorithm

Given an alphabet A with frequency distribution $\{f(a) : a \in A\}$. The binary Huffman tree is constructed using a priority queue, Q , of nodes, with labels (frequencies) as keys.

Huffman(A)

```
{   $n = |A|$ ;  
     $Q = A$ ;                                the future leaves  
    for  $i = 1$  to  $n - 1$                     Why  $n - 1$ ?  
    {   $z = \text{new node}$ ;  
         $left[z] = \text{Extract-Min}(Q)$ ;  
         $right[z] = \text{Extract-Min}(Q)$ ;  
         $f[z] = f[left[z]] + f[right[z]]$ ;  
         $\text{Insert}(Q, z)$ ;  
    }  
    return  $\text{Extract-Min}(Q)$  root of the tree  
}
```

Running time is $O(n \log n)$, as each priority queue operation takes time $O(\log n)$.

Huffman Codes are Optimal

Lemma: Consider the two letters, x and y with the **smallest frequencies**. Then is an optimal code tree in which these two letters are sibling leaves in the tree in the lowest level.

Proof: Let T be an optimum prefix code tree, and let b and c be two siblings at the maximum depth of the tree (must exist because T is full). Assume without loss of generality that $f(b) \leq f(c)$ and $f(x) \leq f(y)$ (if this is not true, then rename these characters). Since x and y have the two smallest frequencies it follows that $f(x) \leq f(b)$ (they may be equal) and $f(y) \leq f(c)$ (may be equal). Because b and c are at the deepest level of the tree we know that $d(b) \geq d(x)$ and $d(c) \geq d(y)$.

Now switch the positions of x and b in the tree resulting in a different tree T' and see how the cost changes. Since T is optimum,

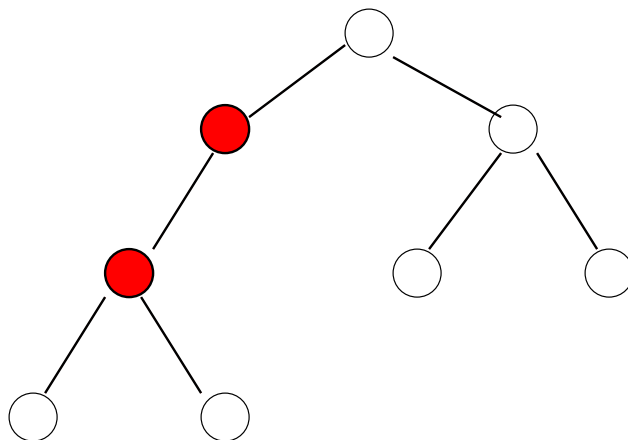
$$\begin{aligned} B(T) &\leq B(T') \\ &= B(T) - f(x)d(x) - f(b)d(b) + f(x)d(b) + f(b)d(x) \\ &= B(T) - (f(b) - f(x))(d(b) - d(x)) \\ &\leq B(T). \end{aligned}$$

Therefore, $B(T') = B(T)$, that is, T' is an optimum tree. By switching y with c we get a new tree T'' which by a similar argument is optimum. The final tree T'' satisfies the statement of the claim.

An Observation: Full Trees

Lemma: The tree for any **optimal prefix code** must be “**full**”, meaning that every internal node has exactly two children.

Proof: If some internal node had only one child then we could simply get rid of this node and replace it with its unique child. This would decrease the total cost of the encoding.



Huffman Codes are Optimal

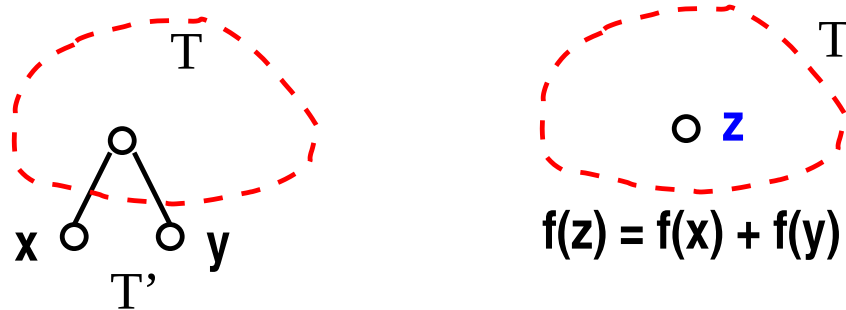
Theorem: Huffman's algorithm produces an optimum prefix code tree.

Proof: By induction on n . When $n = 2$, obvious.

Assume inductively that with strictly fewer than n letters, Huffman's algorithm is guaranteed to produce an optimum tree. We want to show this is also true with exactly n letters.

Huffman Codes are Optimal

Proof – continued: Consider an alphabet A' of n letters. Let T' be an optimum tree for A' with the two letters of lowest frequency x and y as sibling leaves (exists by Lemma). Let T be the coding tree for $A = A' \cup \{z\} - \{x, y\}$ ($n - 1$ leaves) obtained by removing x and y and replacing their parent by z .



$d(x) = d(y) = d(z) - 1$ and $f(z) = f(x) + f(y)$,
so

$$\begin{aligned} B(T) &= B(T') - f(x)d(x) - f(y)d(y) + f(z)d(z) \\ &= B(T') - (f(x) + f(y)). \end{aligned}$$

Huffman Codes are Optimal

Proof – continued: By the induction hypotheses, the Huffman algorithm gives a Huffman code tree H_A that is optimal for A . Let $H_{A'}$ be the tree obtained by adding x and y as children of z in H_A . Note that, as in the calculations for T and T' , we have:

$$B(H_A) = B(H_{A'}) - (f(x) + f(y)).$$

Also, note that, by the optimality of H_A , we have that $B(H_A) \leq B(T)$, so

$$\begin{aligned} B(H_{A'}) &= B(H_A) + f(x) + f(y) \\ &\leq B(T) + f(x) + f(y) \\ &= B(T'). \end{aligned}$$

Since we started with the assumption that T' was an optimal tree for A' this implies that $H_{A'}$, which is exactly the tree constructed by the Huffman algorithm, is also an optimal tree for A' and we are done.

Design and Analysis of Algorithms

CSE 5311

Lecture 17 Greedy algorithms: Huffman Coding

Junzhou Huang, Ph.D.

Department of Computer Science and Engineering

Data Compression

- Suppose we have 1000000000 (1G) character data file that we wish to include in an email.
- Suppose file only contains 26 letters $\{a, \dots, z\}$.
- Suppose each letter a in $\{a, \dots, z\}$ occurs with frequency f_a .
- Suppose we encode each letter by a binary code
- If we use a fixed length code, we need 5 bits for each character
- The resulting message length is $5(f_a + f_b + \dots + f_z)$
- **Can we do better?**

Huffman Coding

- The basic idea
 - Instead of storing each character in a file as an 8-bit ASCII value, we will instead store the more frequently occurring characters using fewer bits and less frequently occurring characters using more bits
 - On average this should decrease the filesize (usually $1/2$)
- Huffman codes can be used to compress information
 - Like WinZip – although WinZip doesn't use the Huffman algorithm
 - JPEGs do use Huffman as part of their compression process

Data Compression: A Smaller Example

- Suppose the file only has 6 letters {a,b,c,d,e,f} with frequencies

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	
.45	.13	.12	.16	.09	.05	
000	001	010	011	100	101	Fixed length
0	101	100	111	1101	1100	Variable length

- Fixed length 3G=3000000000 bits
- Variable length

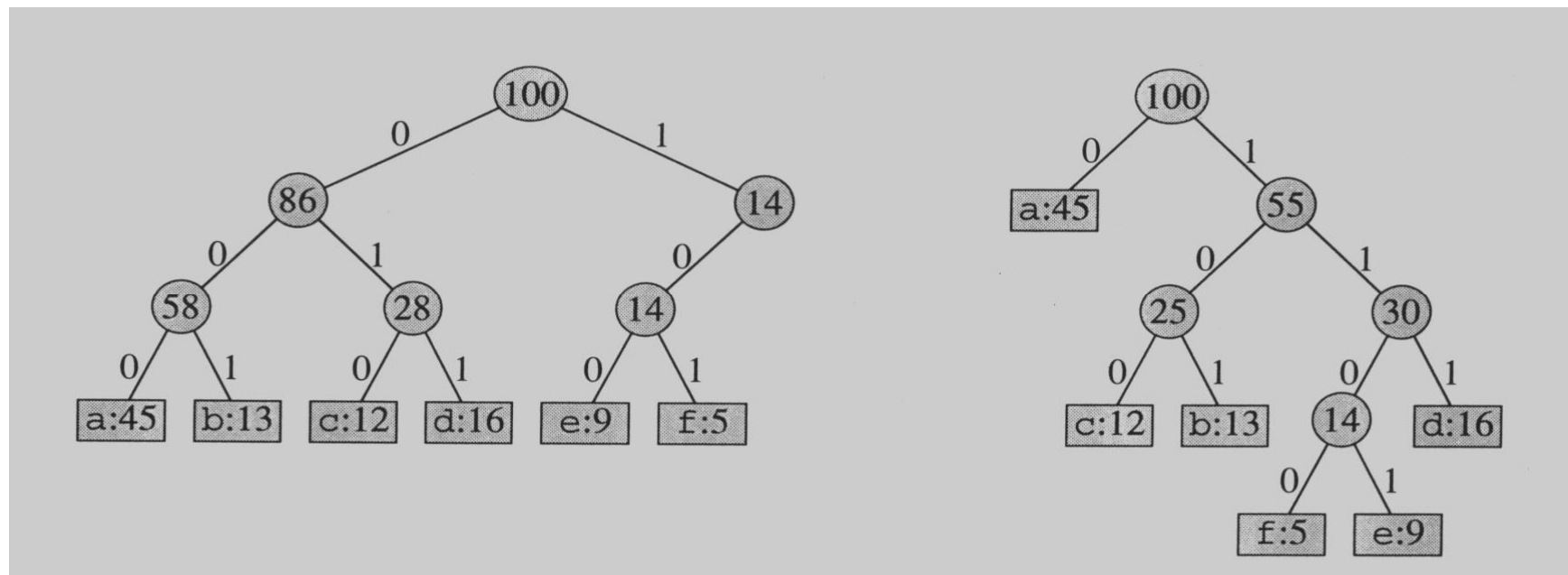
$$(.45 \bullet 1 + .13 \bullet 3 + .12 \bullet 3 + .16 \bullet 3 + .09 \bullet 4 + .05 \bullet 4) = 2.24G$$

How to decode?

- At first it is not obvious how decoding will happen, but this is possible if we use prefix codes

Prefix Codes

- No encoding of a character can be the prefix of the longer encoding of another character, for example, we could not encode t as 01 and x as 01101 since 01 is a prefix of 01101
- By using a binary tree representation we will generate prefix codes provided all letters are leaves



Prefix codes

- A message can be decoded uniquely.
- Following the tree until it reaches to a leaf, and then repeat!
- Draw a few more tree and produce the codes!!!

Some Properties

- Prefix codes allow easy decoding
 - Given a: 0, b: 101, c: 100, d: 111, e: 1101, f: 1100
 - Decode 001011101 going left to right, 0|01011101, a|0|1011101, a|a|101|1101, a|a|b|1101, a|a|b|e
- An optimal code must be a full binary tree (a tree where every internal node has two children)
- For C leaves there are $C-1$ internal nodes
- The number of bits to encode a file is

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

where $f(c)$ is the freq of c , $d_T(c)$ is the tree depth of c , which corresponds to the code length of c

Optimal Prefix Coding Problem

- Input: Given a set of n letters $(c_1, \dots, c_n)$ with frequencies $(f_1, \dots, f_n)$.
- Construct a full binary tree T to define a prefix code that **minimizes** the average code length

$$\text{Average}(T) = \sum_{i=1}^n f_i \bullet \text{length}_T(c_i)$$

Greedy Algorithms

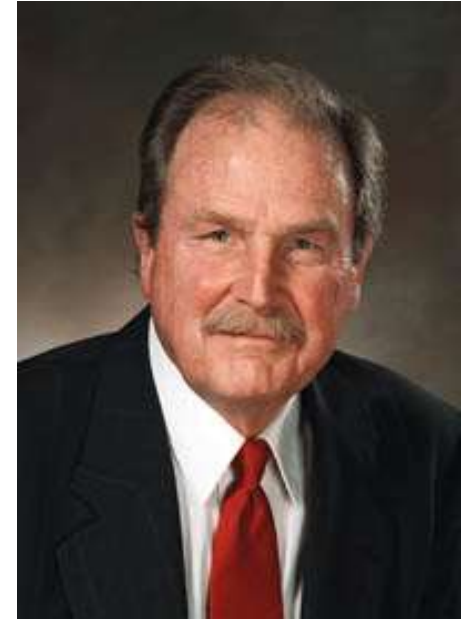
- Many optimization problems can be solved using a greedy approach
 - The basic principle is that local optimal decisions may be used to build an optimal solution
 - But the greedy approach may not always lead to an optimal solution overall for all problems
 - The key is knowing which problems will work with this approach and which will not
- We will study
 - **The problem of generating Huffman codes**

Greedy algorithms

- A *greedy algorithm* always makes the choice that looks best at the moment
 - My everyday examples:
 - Driving in Los Angeles, NY, or Boston for that matter
 - Playing cards
 - Invest on stocks
 - Choose a university
 - The hope: a locally optimal choice will lead to a globally optimal solution
 - For some problems, it works
- Greedy algorithms tend to be easier to code

David Huffman's idea

- A Term paper at MIT



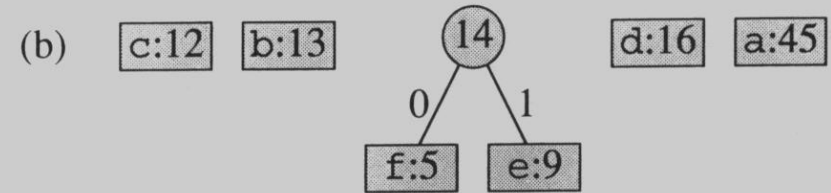
- Build the tree (code) bottom-up in a greedy fashion
- Origami aficionado

Story behind

- David Huffman was a student in an EE course in 1951. His professor, Robert Fano, offered students a choice of taking a final exam or writing a term paper.
- Huffman did not want to take the final so he started working on the term paper. The topic of the paper was to find the most efficient (optimal) code.
- The fact: it was an open research problem
- Huffman spent a lot of time on the problem and was ready to give up when the solution suddenly came to him. It had the lowest possible average message length.
- Later Huffman said that likely he would not have even attempted the problem if he had known that it was an open research problem

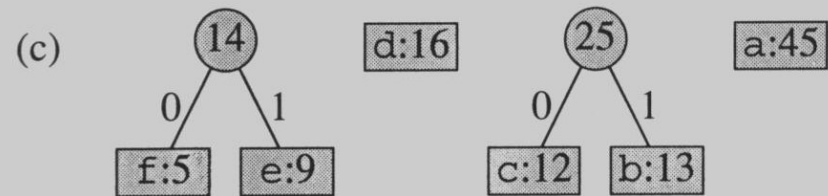
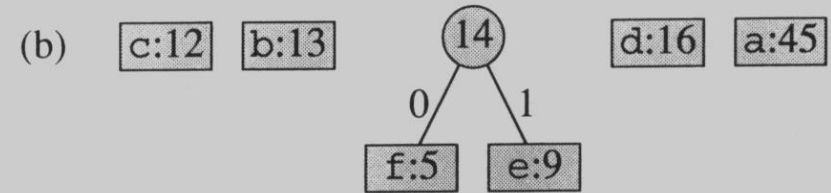
Building the Encoding Tree

(a) f:5 e:9 c:12 b:13 d:16 a:45 \



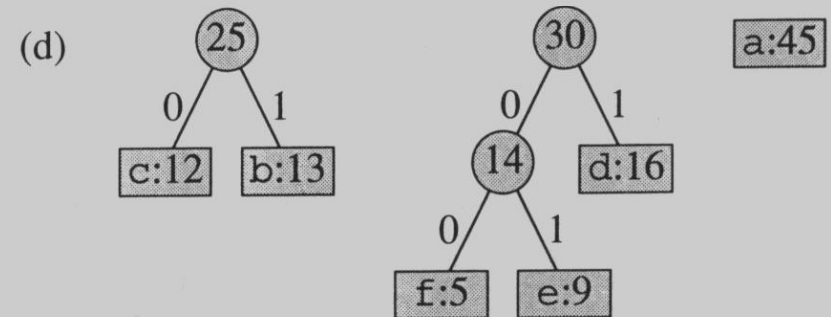
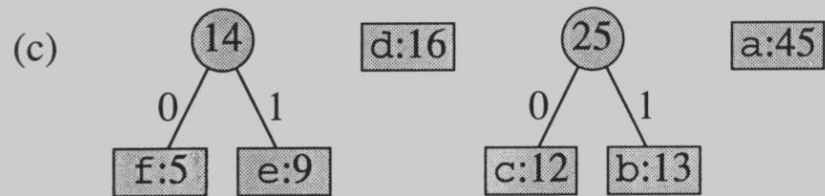
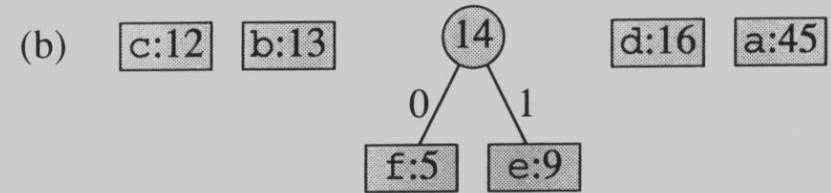
Building the Encoding Tree

(a) f:5 e:9 c:12 b:13 d:16 a:45 \



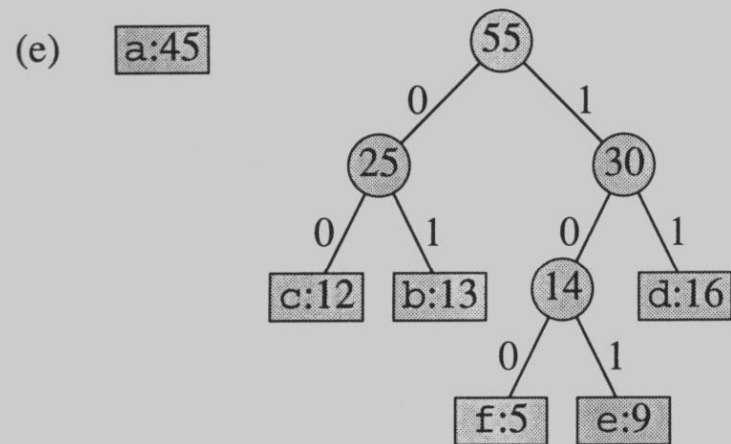
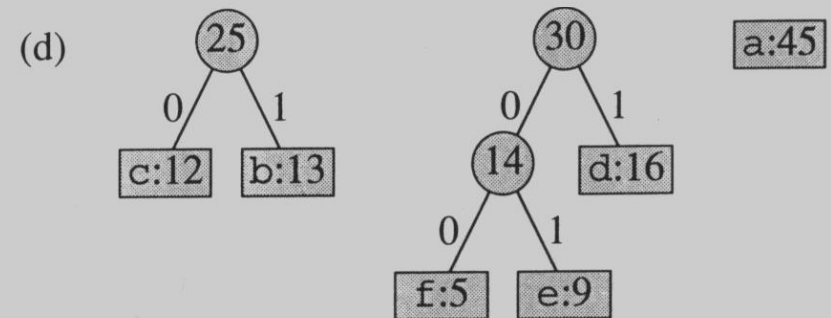
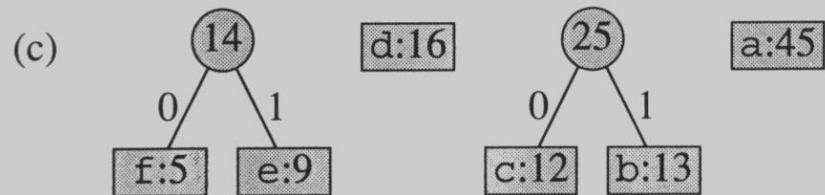
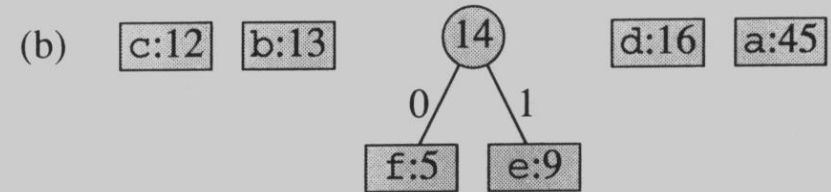
Building the Encoding Tree

(a) f:5 e:9 c:12 b:13 d:16 a:45



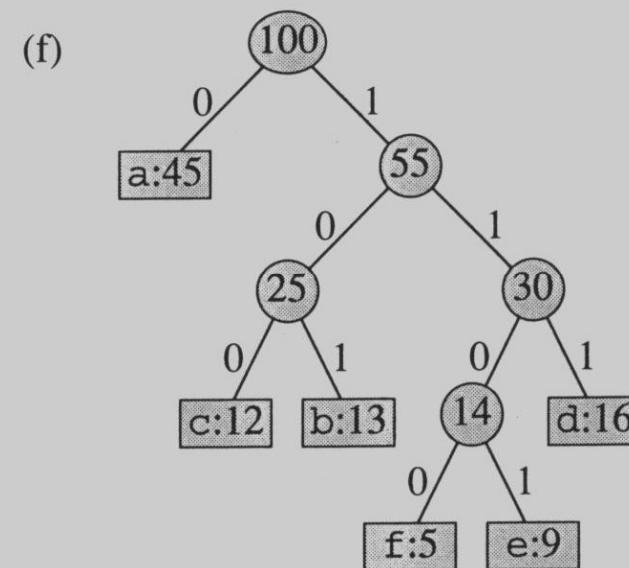
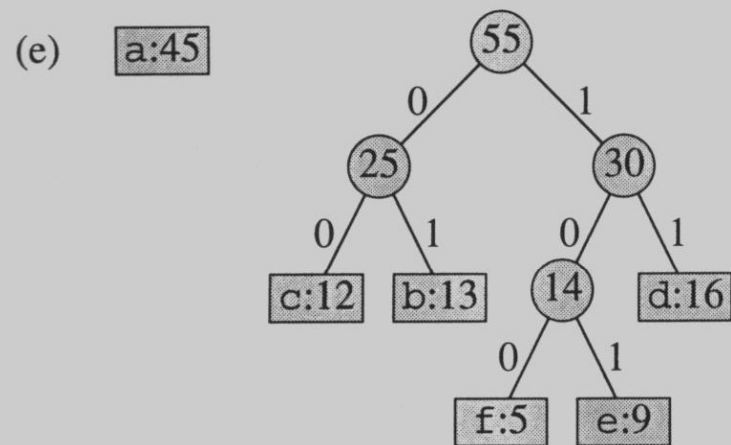
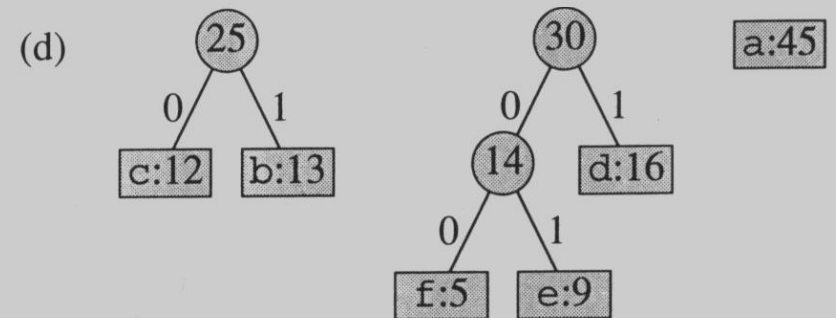
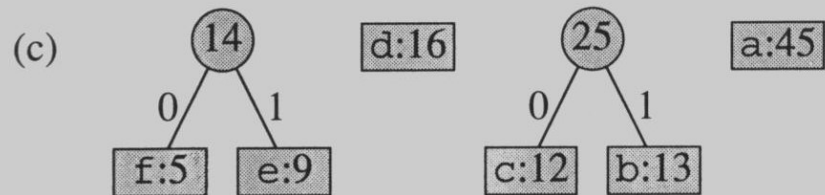
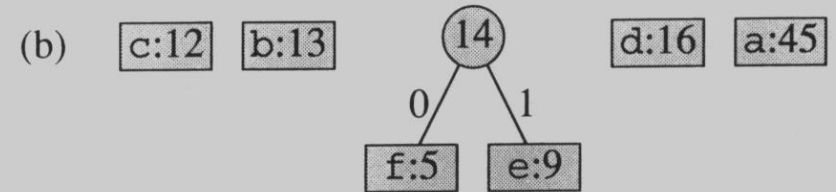
Building the Encoding Tree

(a) f:5 e:9 c:12 b:13 d:16 a:45



Building the Encoding Tree

(a) f:5 e:9 c:12 b:13 d:16 a:45



The Algorithm

HUFFMAN(C)

```
1   $n \leftarrow |C|$ 
2   $Q \leftarrow C$ 
3  for  $i \leftarrow 1$  to  $n - 1$ 
4      do allocate a new node  $z$ 
5           $left[z] \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$ 
6           $right[z] \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$ 
7           $f[z] \leftarrow f[x] + f[y]$ 
8           $\text{INSERT}(Q, z)$ 
9  return  $\text{EXTRACT-MIN}(Q)$  ▷ Return the root of the tree.
```

Q :priority queue

—

- An appropriate data structure is a binary min-heap
- Rebuilding the heap is $\lg n$ and $n-1$ extractions are made, so the complexity is $O(n \lg n)$
- The encoding is NOT unique, other encoding may work just as well, but none will work better

Correctness of Huffman's Algorithm

Lemma 16.2

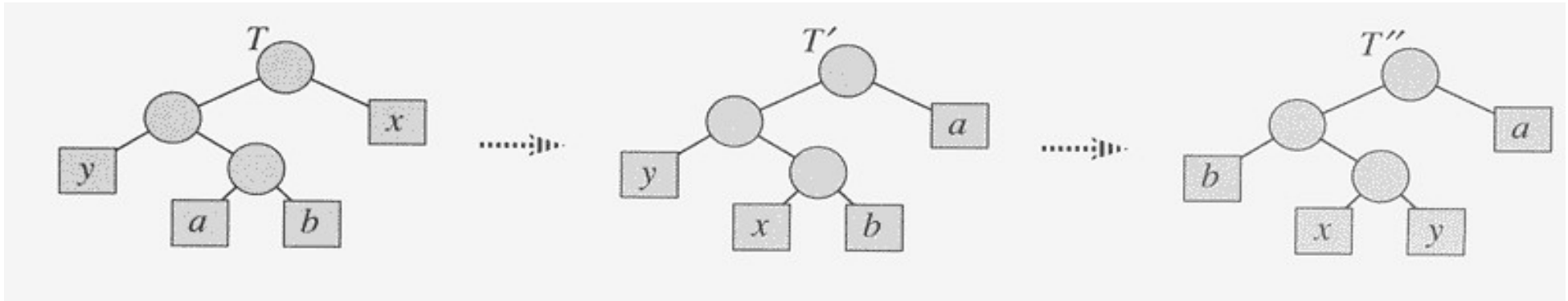
Let C be an alphabet in which each character $c \in C$ has frequency $f[c]$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.

Proof:

The idea of the proof is to take the tree T representing an arbitrary optimal prefix code and modify it to make a tree representing another optimal prefix code such that the characters x and y appear as sibling leaves of maximum depth in the new tree.

If we can construct such a tree, then the codewords for x and y will have the same length and differ only in the last bit.

Lemma 16.2



An illustration of the key step in the proof of Lemma 16.2.

In the optimal tree T , leaves a and b are two siblings of **maximum depth**. Leaves x and y are the two characters with **the lowest frequencies**; they appear in arbitrary positions in T .

Assuming that $x \neq b$, swapping leaves a and x produces tree T' , and then swapping leaves b and y produces tree T'' .

Since each swap does not increase the cost, the resulting tree T'' is also an optimal tree.

Lemma 16.2

- Let a and b be two characters that are sibling leaves of maximum depth in T . Assume $f[a] \leq f[b]$ and $f[x] \leq f[y]$
- Since $f(x)$ and $f(y)$ are the two lowest leaf frequencies, we assume $f[x] < f[y]$. Therefore, we have $f[x] \leq f[a]$; $f[y] \leq f[b]$
- As shown in the previous page, we exchange the positions in T of a and x to produce a tree T' , and then we exchange the positions in T' of b and y to produce a tree T'' in which x and y are sibling leaves of maximum depth.
- The cost difference between T and T' is

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c) d_T(c) - \sum_{c \in C} f(c) d_{T'}(c) \\ &= f[x] d_T(x) + f[a] d_T(a) - f[x] d_{T'}(x) - f[a] d_{T'}(a) \\ &= f[x] d_T(x) + f[a] d_T(a) - f[x] d_T(a) - f[a] d_T(x) \\ &= (f[a] - f[x])(d_T(a) - d_T(x)) \\ &\geq 0 \quad \text{Why?} \end{aligned}$$

Lemma 16.2

- Term 1: $f[a]-f[x] \geq 0$: x is a minimum-frequency leaf
- Term 2: $d_T(a) - d_T(x) \geq 0$: a is a leaf of maximum depth in T
- Therefore, $B(T)-B(T')=(f[a]-f[x])(d_T(a) - d_T(x)) \geq 0$
- Similarly, exchanging y and b does not increase the cost, then, $B(T')-B(T'') \geq 0$

$$B(T'') \leq B(T),$$

but T is optimal, $B(T) \leq B(T'')$

$$\Rightarrow B(T'') = B(T)$$

Therefore T'' is an optimal tree in which x and y appear as sibling leaves of maximum depth

Correctness of Huffman's Algorithm

Lemma 16.3

Let C be a given alphabet with frequency $f[c]$ defined for each character $c \in C$. Let x and y be two characters in C with minimum frequency. Let C' be the alphabet C with characters x, y removed and (new) character z added, so that $C' = C - \{x, y\} \cup \{z\}$; define f for C' as for C , except that $f[z] = f[x] + f[y]$. Let T' be any tree representing an optimal prefix code for the alphabet C' . Then the tree T , obtained from T' by replacing the leaf node for z with an internal node having x and y as children, represents an optimal prefix code for the alphabet C .

- We first show that: $B(T) = B(T') + f[x] + f[y] \Rightarrow B(T') = B(T) - f[x] - f[y]$

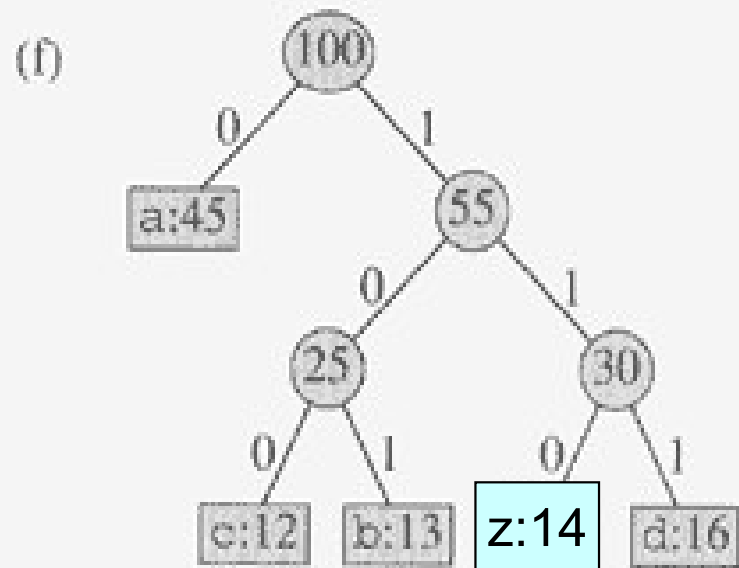
$$\text{--For each } c \in C - \{x, y\} \Rightarrow d_T(c) = d_{T'}(c) \Rightarrow f[c]d_T(c) = f[c]d_{T'}(c)$$

$$\text{--}d_T(x) = d_T(y) = d_{T'}(z) + 1$$

$$\text{--}f[x]d_T(x) + f[y]d_T(y) = (f[x] + f[y])(d_{T'}(z) + 1)$$

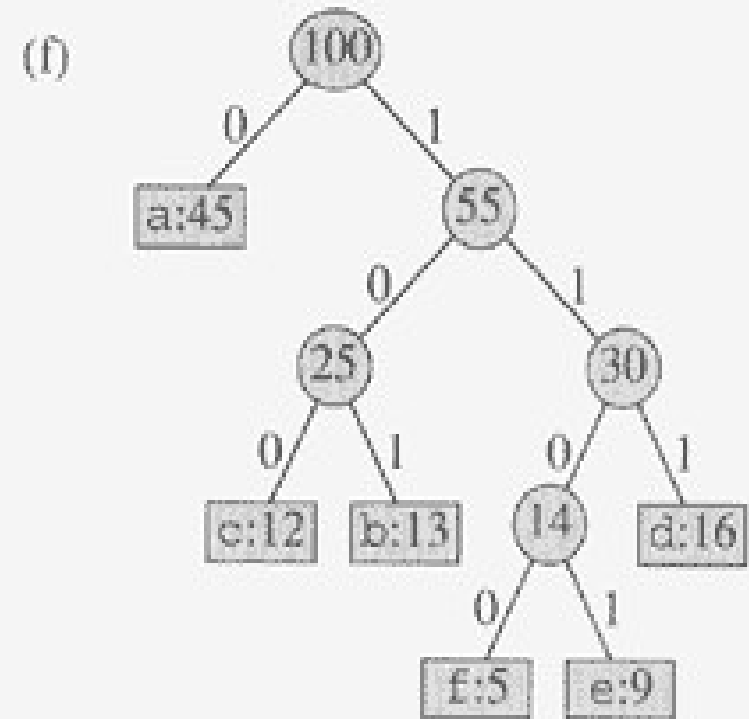
$$\text{--} = f[z]d_{T'}(z) + (f[x] + f[y])$$

$$B(T') = B(T) - f[x] - f[y]$$



$$B(T') = 45*1 + 12*3 + 13*3 + (5+9)*3 + 16*3$$

$$= B(T) - 5 - 9$$



$$B(T) = 45*1 + 12*3 + 13*3 + 5*4 + 9*4 + 16*3$$

Proof of Lemma 16.3

Prove by contradiction

- Suppose that T does not represent an optimal prefix code for C . Then there exists a tree T'' such that $B(T'') < B(T)$.
- Without loss of generality (by Lemma 16.2), T'' has x and y as siblings. Let T''' be the tree T'' with the common parent x and y replaced by a leaf with frequency $f[z] = f[x] + f[y]$.
- Then
$$\begin{aligned} B(T''') &= B(T'') - f[x] - f[y] \\ &< B(T) - f[x] - f[y] \\ &= B(T) \end{aligned}$$
 - T''' is better than T' $\rightarrow$ contradiction to the assumption that T' is an optimal prefix code for C' . Thus, T must represent an optimal prefix code for the alphabet C .

Example: Huffman Coding

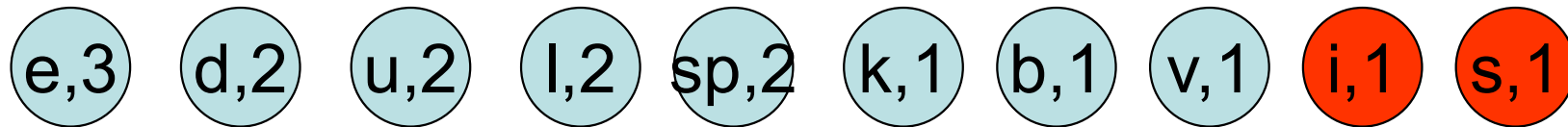
- As an example, let's take the string:
“duke blue devils”
- We first do a frequency count of the characters:
➤ e:3, d:2, u:2, l:2, space:2, k:1, b:1, v:1, i:1, s:1
- Next we use a Greedy algorithm to build up a Huffman Tree
 - We start with nodes for each character



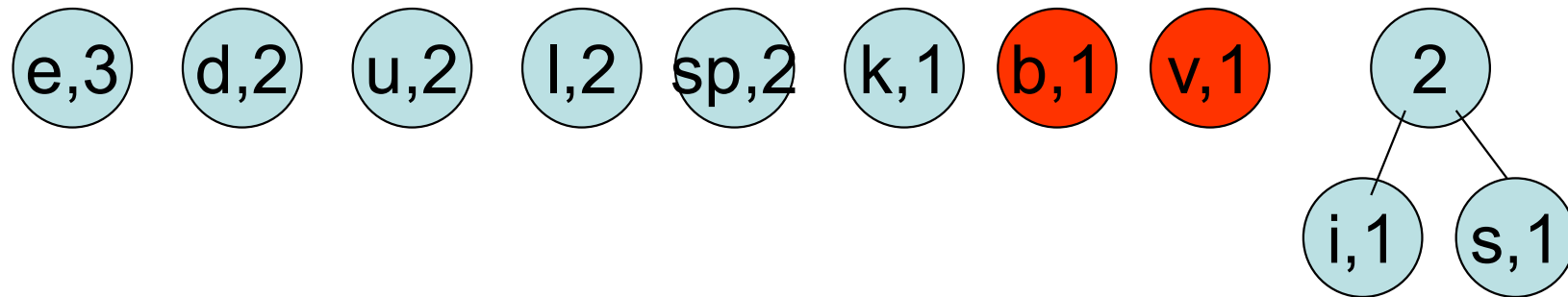
Example: Huffman Coding

- We then pick the nodes with the smallest frequency and combine them together to form a new node
 - The selection of these nodes is the Greedy part
- The two selected nodes are removed from the set, but replace by the combined node
- This continues until we have only 1 node left in the set

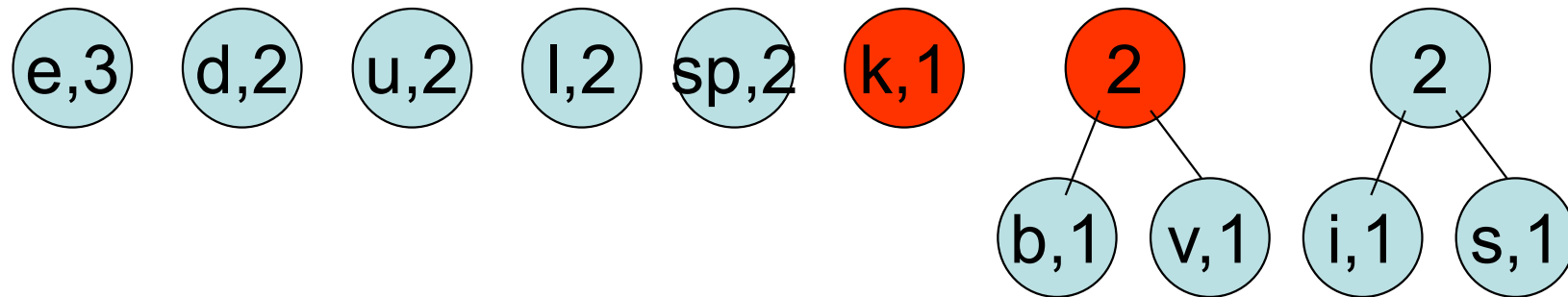
Example: Huffman Coding



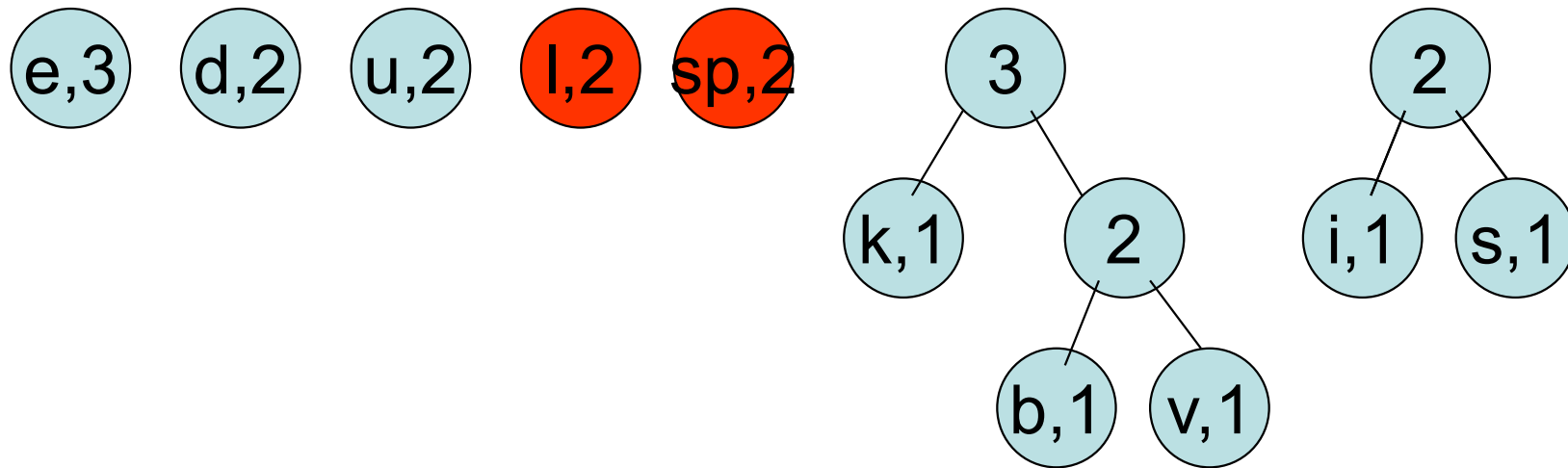
Example: Huffman Coding



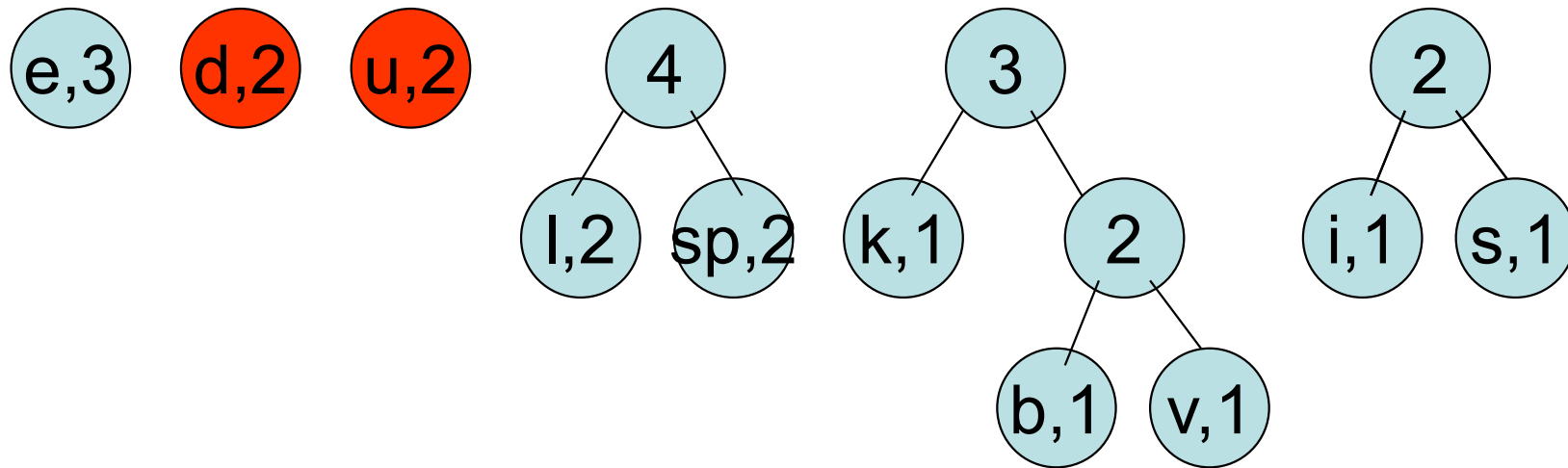
Example: Huffman Coding



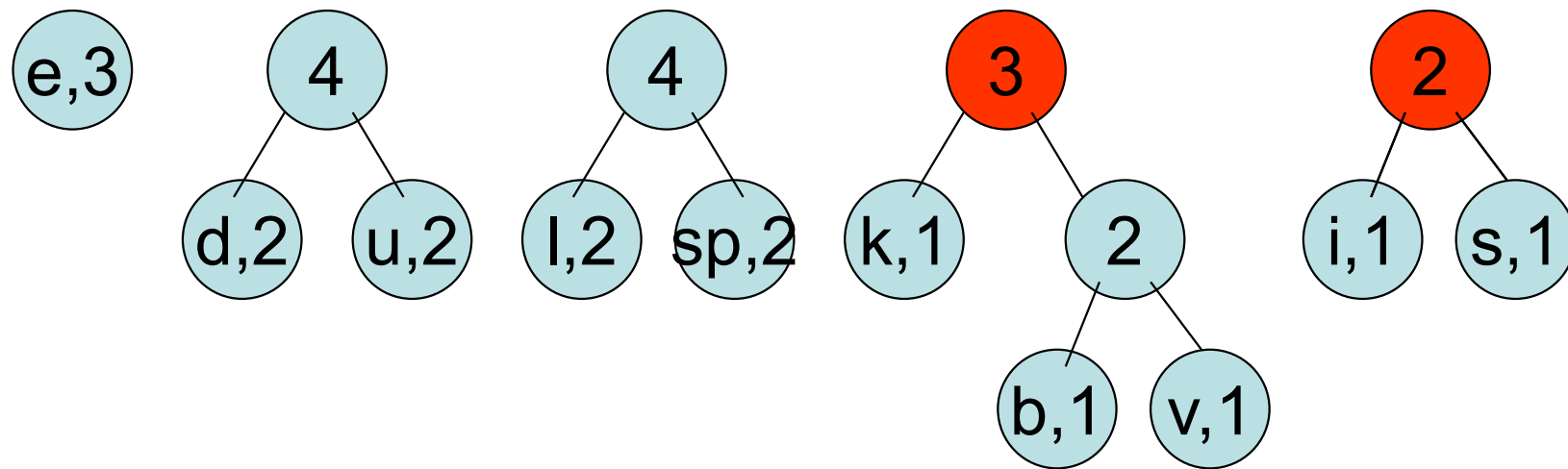
Example: Huffman Coding



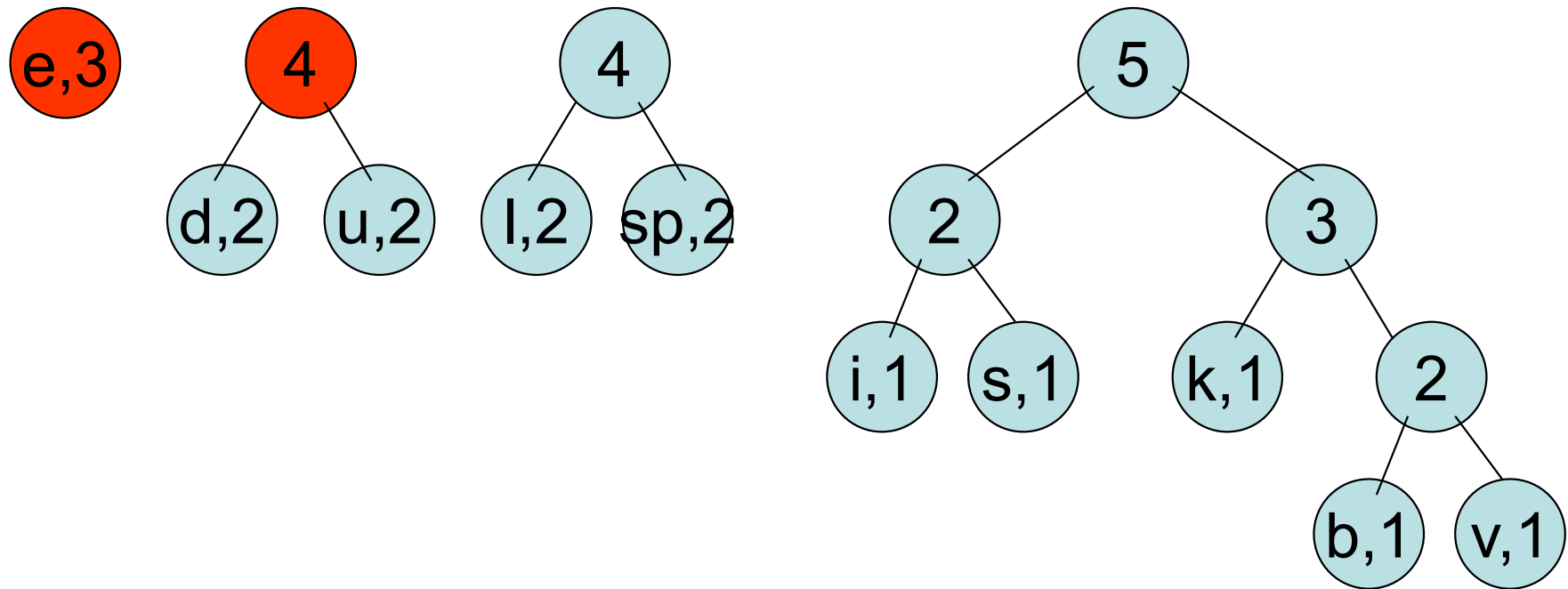
Example: Huffman Coding



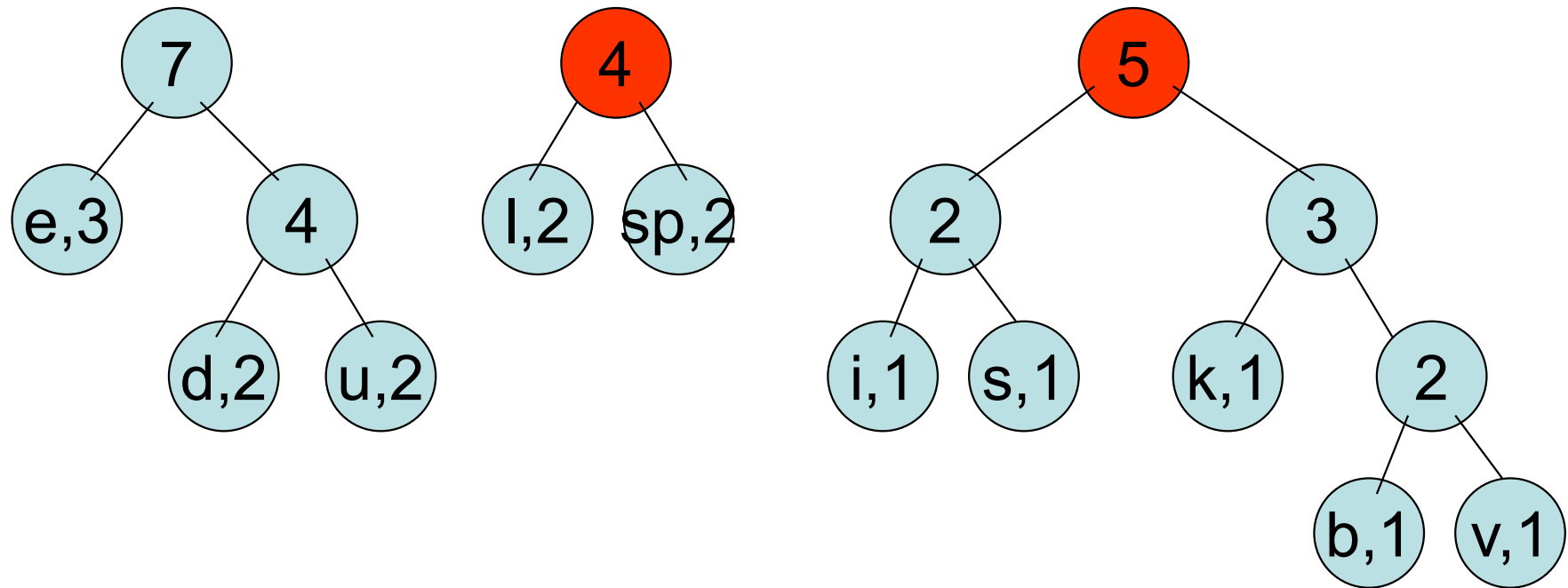
Example: Huffman Coding



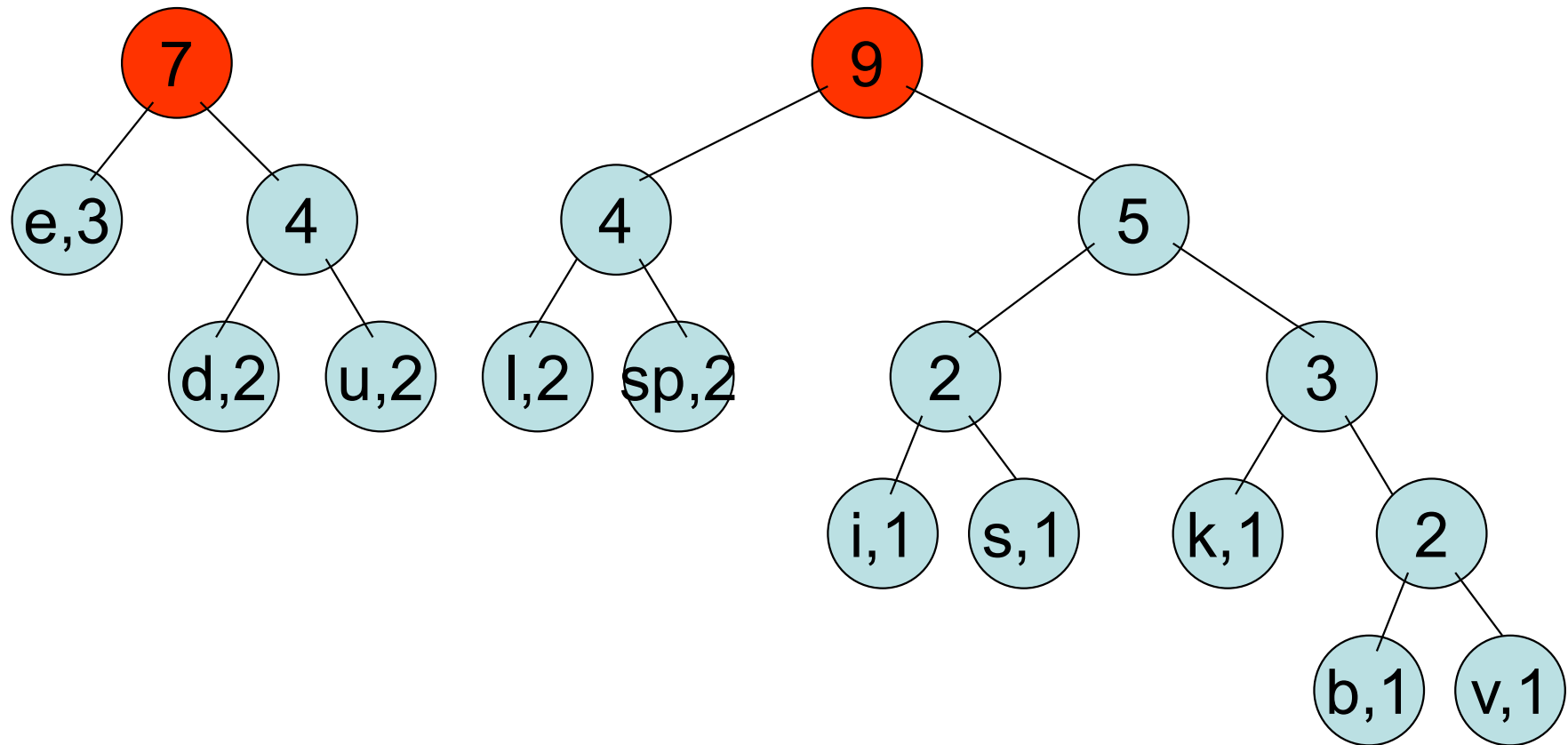
Example: Huffman Coding



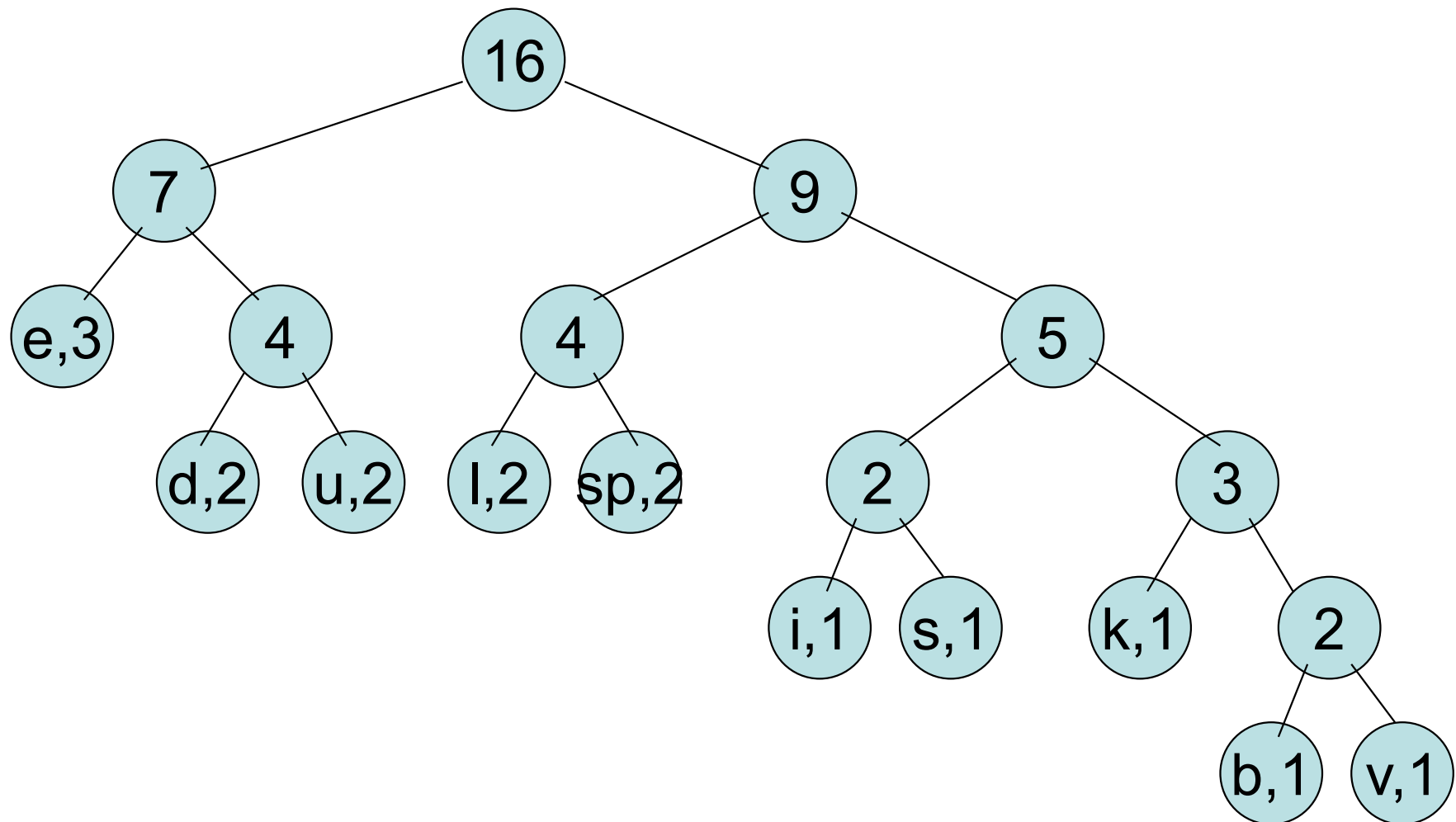
Example: Huffman Coding



Example: Huffman Coding



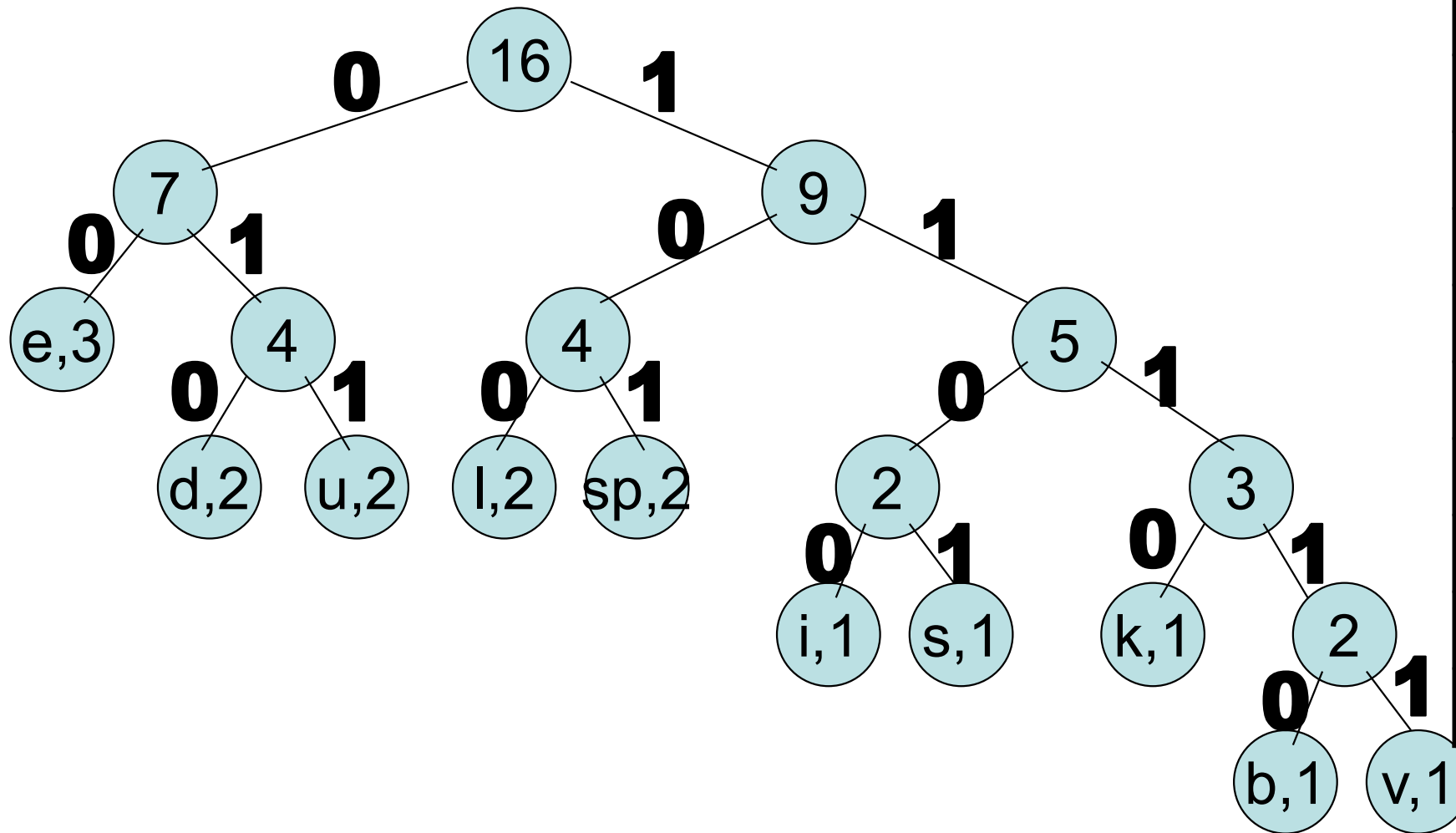
Example: Huffman Coding



Example: Huffman Coding

- Now we assign codes to the tree by placing a 0 on every left branch and a 1 on every right branch
- A traversal of the tree from root to leaf give the Huffman code for that particular leaf character
- Note that no code is the prefix of another code

Example: Huffman Coding



e	00
d	010
u	011
l	100
sp	101
i	1100
s	1101
k	1110
b	11110
v	11111

Example: Huffman Coding

- These codes are then used to encode the string
- Thus, “duke blue devils” turns into:

010 011 1110 00 101 11110 100 011 00 101 010 00 11111 1100 100 1101

- When grouped into 8-bit bytes:

01001111 10001011 11101000 11001010 10001111 11100100 1101xxxx

- Thus it takes 7 bytes of space compared to 16 characters *
1 byte/char = 16 bytes uncompressed

Example: Huffman Coding

- Uncompressing works by reading in the file bit by bit
 - Start at the root of the tree
 - If a 0 is read, head left
 - If a 1 is read, head right
 - When a leaf is reached decode that character and start over again at the root of the tree
- Thus, we need to save Huffman table information as a header in the compressed file
 - Doesn't add a significant amount of size to the file for large files (which are the ones you want to compress anyway)
 - Or we could use a fixed universal set of codes/frequencies

Exercise

- Prefix codes
 - Given a: 0, b: 101, c: 100, d: 111, e: 1101, f: 1100
 - Decode 001011101 going left to right
- Answer
 - 001011101 $\rightarrow$ 0|01011101
 - a|0|1011101 $\rightarrow$ a|a|101|1101
 - a|a|b|1101 $\rightarrow$ a|a|b|e

Exercise

- Prefix codes
 - Given I: 0, L: 101, Y: 100, X: 111, T: 1101, Z: 1100
 - Decode 01011001101 going left to right
- Answer
 - 01011001101 $\rightarrow$ 0|1011001101
 - I|101|1001101 $\rightarrow$ I|L|100|1101
 - I|L|Y|1101 $\rightarrow$ I|L|Y|T

Next

- Graph Algorithms
 - BFS & DFS
 - Topological Sort
 - Minimum Spanning Trees
 - Single Source Shortest Path
 - All-pairs Shortest Path
 - Maximum Flow
- Important
 - Please read Lecture/Textbook first